

Day 7 — Mastering Signed Bits

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~2 hours (≈ 20 minutes retrieval, ≈ 80 minutes solving and coding, ≈ 20 minutes explanation)

Today one byte will play three roles: an unsigned number, a signed number, and a device register. Your job is to keep the interpretations separate.

1. The byte under investigation

Here is the only pattern we need:

```
11010110
```

Do not calculate yet. First list the questions we could ask:

- What is its unsigned value?
- What is its signed two's-complement value?
- If it is a register, which flags are set?
- If bits 5:3 form a field, what field value do they encode?
- What happens if we add one?
- What happens if we widen it?
- Is any one interpretation “inside” the byte?

One pattern, many rules. This is Day 2's principle applied to Day 4-6 tools.

2. Retrieval map

On a blank page, draw four boxes:

SIGNED REPRESENTATION	BITWISE OPERATIONS
FIXED-WIDTH ARITHMETIC	WIDTH CHANGES

Fill them from memory:

Signed representation

- signed magnitude and its two-zero problem,
- one's complement and its two-zero problem,
- two's complement as invert-plus-one or modulo 2^n ,
- the range -2^{n-1} through $2^{n-1}-1$.

Bitwise operations

- AND selects or clears,
- OR sets,
- XOR toggles,
- NOT inverts,
- shifts position fields.

Fixed-width arithmetic

- unsigned modulo behavior,
- carry for unsigned range failure,
- overflow for signed range failure,
- signed overflow in C is undefined behavior.

Width changes

- zero extension preserves unsigned value,
- sign extension preserves signed value,
- narrowing discards high bits and must be reasoned about carefully.

Only reopen earlier PDFs after attempting the map.

3. Decode the pattern as two numbers

Unsigned

bits:	1	1	0	1	0	1	1	0
place:	128	64	32	16	8	4	2	1

Add active place values.

Signed

The high bit is 1, so under eight-bit two's complement it is negative.

Use either:

```
signed value = unsigned value - 256
```

or:

```
invert → add one → attach negative sign
```

Perform both methods. If they disagree, one arithmetic step is wrong.

Why is subtracting 256 valid? Because signed and unsigned readings choose different representatives of the same residue modulo 2^8 .

4. Decode the same pattern as a register

Use this fictional device layout:

bit:	7	6	5	4	3	2	1	0
field:	READY	ERROR	MODE[2:0]		RSV	IRQ_EN	START	

For 11010110, answer:

1. Is READY set?
2. Is ERROR set?
3. What is MODE?
4. Is IRQ_EN set?
5. Is START set?

Now write masks:

```
#define READY_MASK ...
#define ERROR_MASK ...
#define MODE_MASK ...
#define MODE_SHIFT ...
#define IRQ_EN_MASK ...
#define START_MASK ...
```

Then write expressions to:

- test ERROR,
- clear IRQ_EN,
- toggle START,
- extract MODE,
- replace MODE with 3 while preserving all other bits.

The register interpretation does not care that the same pattern could mean unsigned 214 or signed -42. Those are different questions.

5. Arithmetic interpretation

Add these to the original byte, one at a time:

```
00000001
00110000
10000000
```

For each:

1. compute the low eight result bits,
2. decide whether unsigned carry occurred,
3. decide whether signed overflow occurred,
4. interpret the result both signed and unsigned.

Do not use the numeric values alone. Perform the binary addition and observe the carry into and out of the high bit.

The goal is not speed. It is learning to ask “overflow under which interpretation?”

6. Widening interpretation

Extend `11010110` to 16 bits in two ways:

Zero extension

```
00000000 11010110
```

Sign extension

```
11111111 11010110
```

Now calculate each 16-bit numeric value.

The low eight bits are identical. The high eight bits encode our decision about which original value must be preserved.

This gives a useful rule:

Extension is not just adding storage. It is preserving a chosen interpretation while changing width.

7. Integrated C lab

Day 5 provided:

```
./daily/challenges/day-005-register-mask-lab.c
```

Day 6 provided:

```
./daily/challenges/day-006-overflow-lab.c
```

Complete the register-mask TODOs first. Then build both programs with warnings enabled.

```
gcc -std=c17 -Wall -Wextra -Wconversion -O0 -g \  
./daily/challenges/day-005-register-mask-lab.c \  
-o /tmp/register-mask-lab  
  
gcc -std=c17 -Wall -Wextra -O0 -g \  
./daily/challenges/day-006-overflow-lab.c \  
-o /tmp/overflow-lab
```

For the overflow program, repeat with UBSan:

```
gcc -std=c17 -Wall -Wextra -O1 -g \  
-fsanitize=undefined -fno-sanitize-recover=undefined \  
./daily/challenges/day-006-overflow-lab.c \  
-o /tmp/overflow-lab-ubsan
```

Record:

- what you predicted,
- what the compiler warned about,
- what ordinary execution printed,
- where UBSan stopped,
- and which C rule explains each observation.

8. A realistic register problem

A storage controller reports:

```
STATUS = 0xD6
```

The manual states:

```
bit 7    operation complete
bit 6    unrecoverable error
bits 5:3  retry count
bit 2    reserved
bit 1    interrupt pending
bit 0    controller busy
```

Write a short diagnosis in plain language.

Then suppose software acknowledges the interrupt by writing a 1 to bit 1 (write-one-to-clear semantics). Would ordinary read-modify-write necessarily be safe?

This is a preview of a driver lesson: register bits are not always ordinary memory. Some have side effects on read or write. The mask calculation may be correct while the access sequence is wrong.

Mechanism and semantics must both come from the hardware specification.

9. The mastery checkpoint

The curriculum asks:

Given an eight-bit pattern, explain all plausible interpretations and identify the interpretation rule.

“All plausible” does not mean guessing every format ever invented. It means showing that you know how to ask for context.

For `11010110`, a strong answer says:

- As unsigned binary, use positive powers-of-two place values.
- As eight-bit two’s complement, the high bit has negative weight, producing a negative value.
- As a packed register, consult the register layout and decode each flag/field.

- As text, consult a character encoding; one byte may not even represent a complete character in a variable-length encoding.
- As machine code, use the selected ISA and instruction-boundary context.
- As pixels, use image dimensions, pixel format, bit order, and color rules.

Then state the governing principle:

The bits constrain the possibilities, but the format, type, ISA, or device specification selects the interpretation.

10. Explain it to your kernel-engineer self

Connect the foundations to familiar work:

- A page-table entry is a machine word interpreted as address bits plus present/write/user/ accessed/dirty and other flags.
- An MSR or device register is a word interpreted according to a vendor specification.
- `cpumask_t` uses bits as membership flags.
- Error codes pack independent facts into bit positions.
- Sign extension matters when register values move between operand widths.
- Overflow reasoning matters when sizes, offsets, page counts, and allocation lengths are calculated.

None of those are separate tricks. They express the same foundational idea: patterns become useful when a precise interpretation is applied.

11. Where this is taking us: from calculation to a CPU

We can now state what is still missing.

Bits give us representation. Boolean operations let circuits transform those bits. An adder gives us one useful transformation. But a calculator wired to perform one operation is not yet a general-purpose computer.

To reach a CPU, designers had to solve several additional problems:

1. Where do inputs and intermediate results live?

We need storage elements—eventually registers and memory—so one operation can use the result of an earlier operation.

2. How does the machine choose among operations?

An arithmetic and logic unit may support ADD, AND, OR, compare, and shifts. A control mechanism must select the desired path.

3. How does it know what happens next?

Early machines might be rewired or controlled by switches, plugboards, punched cards, or paper tape. That works, but setup is slow and the control sequence remains physically separate from ordinary data.

4. What if instructions are encoded as patterns too?

This is the stored-program leap. Give operations numeric codes, store those codes in addressable memory, and build control circuitry that repeatedly:

```
fetch instruction pattern from memory
  ↓
decode which operation it requests
  ↓
select registers and ALU function
  ↓
execute and store result
  ↓
advance or change the next-instruction address
```

The Manchester “Baby” demonstrated execution of a program from writable electronic memory in 1948. Historical credit for the stored-program idea is distributed across multiple people and projects; what matters technically is the new abstraction: **instructions and data can both be encoded bit patterns held in memory.**

5. Where does an instruction set come from?

CPU designers define an ISA: a contract assigning instruction meanings to encodings and defining visible registers, operations, and behavior.

bit pattern in memory		ISA rule		requested operation
-----	+	-----	→	-----
instruction encoding		decoder contract		add, load, branch...

The specification is a document/contract. The bytes are stored in memory. The decoder and execution machinery are implemented physically in the CPU. Do not collapse those three locations into “the instruction set is stored in the processor.”

Does the computer think?

At this physical level, no mysterious thinking occurs. Gates respond to input states, registers preserve states, and control circuitry causes an encoded sequence of transitions.

Complex behavior arises because:

- simple operations compose into algorithms,
- algorithms manipulate rich representations,
- software layers build abstractions,
- and modern AI combines enormous computation with learned parameters.

Calling the final behavior “reasoning” may be useful at a higher level, but it does not replace the lower-level causal story. This course will keep both levels visible.

Our next major construction path is therefore:

```
transistor as switch
  → logic gate
  → combinational circuits
  → adder and ALU
  → latch/register
  → control and clock
  → instruction encoding
  → fetch/decode/execute
  → minimal CPU
```

We will build every arrow rather than jumping to the finished processor.

12. Completion rubric

You are ready for the next topic when you can:

- derive a negative two's-complement value rather than recognize it vaguely,
- explain the asymmetric signed range,
- construct and apply set/clear/toggle/test masks,
- extract and replace a packed field,
- distinguish carry from signed overflow,
- explain unsigned versus signed C overflow,
- and choose zero or sign extension for a reason.

If one item is weak, the next day should target that item. Do not repeat everything.

Update `DAILY_PROGRESS.md` with the exact stopping point and your own confidence.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, book pages 21–36.
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 2, §§2.1–2.3.
- Current Linux kernel `include/linux/bits.h`, `include/linux/bitfield.h`, and overflow helpers as modern examples of these foundations. Their APIs will be studied in context later.
- Computer History Museum, "The Stored Program," including the Manchester "Baby" and its 1948 stored-program demonstration: <https://www.computerhistory.org/revolution/birth-of-the-computer/4/87>
- Nokia Bell Labs, history of the transistor and its original communications-engineering motivation: <https://www.bell-labs.com/about/awards/1956-nobel-prize-physics/>

After you work through Day 7: share your results and questions. The next lesson will be generated from the evidence—either a targeted repair day or the next curriculum topic.